DEPARTMENT OF COMPUTER AND INFORMATION SYSTEMS ENGINEERING
BACHELORS IN COMPUTER SYSTEMS ENGINEERING
Course Code and Title: CS-323 Artificial Intelligence
Complex Engineering Problem
TE Batch 2024, Fall Semester 2026
TERM PROJECT

Instructors: Dr. Maria Waqas (Theory), Dr. Sumayya Zafar (Practical)

Course Learning Outcomes (CLOs)

- **CLO 2:** Solve problems using artificial intelligence techniques.
  (Taxonomy level C3, PLO-3 Design / Development of Solutions).
- **CLO 3:** Demonstrate the use of modern tools and techniques for developing intelligent systems.
  (Taxonomy level C3, PLO-5 Modern Tool Usage).

Complex problem-solving attributes (CPA) covered (as per PEC - OBA manual – 2019)

- **CPA-1 Depth of analysis required:** Have no obvious solution and require abstract thinking, originality in analysis to formulate suitable models.
- **CPA-2 Level of interaction:** Require resolution of significant problems arising from interactions between wide ranging or conflicting technical, engineering or other issues.
- **CPA-3 Familiarity:** Can extend beyond previous experiences by applying principles-based approaches.

Students may work in groups of four.

1. Term Project Title

Intelligent Vacuum-Cleaning Robot

2. Problem Definition

Design and implement an intelligent vacuum-cleaning robot that operates in a grid-based environment. The robot must navigate through the environment, visit and clean all specified dirty locations, and return to the charging station. The environment contains:

- **Charging station:** The starting and ending location of the robot.
- **Free cells:** Cells through which the robot can move.
- **Obstacles:** Cells that cannot be traversed.
- **Dirty locations:** Locations that must be visited and cleaned.
- **Movement actions:** The robot can move up, down, left, or right. Diagonal movement is not allowed.

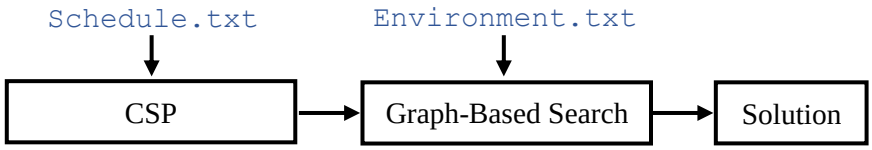
The project requires students to solve different aspects of the problem using two AI approaches:

- **Graph-Based Search:** Finds a valid path from the robot's current location to a specified destination while avoiding obstacles.
- **Constraint Satisfaction Problem (CSP):** Determines the order in which cleaning tasks should be performed while satisfying the specified scheduling constraints.

The CSP solver will generate a feasible cleaning schedule, and the graph-based search component will determine the navigation paths required to execute that schedule. The two components must be integrated to simulate the complete cleaning task, including the robot's return to the charging station. Students must compare the implemented graph-search algorithms using their returned paths, path costs, and numbers of expanded nodes. The integrated simulation must also report the complete journey and its total navigation cost.

The environment for graph-based search must be loaded from `Environment.txt`, while the scheduling constraints for the CSP must be loaded from `Schedule.txt`. The formats of both files are described in the following sections.

The program flow is shown in the following block diagram.



During assessment, different valid environment and scheduling files may be provided for testing. Therefore, the program must load and process the input files according to the specified formats rather than relying on a hardcoded environment, number of tasks, time slots, or constraints.

3. Sample Environment

The sample environment is provided to help students understand the problem and the input file format. It is also suitable for initial testing.

The sample environment (defined in `Environment_Sample.txt`) is an 8 × 8 grid with one charging station, six obstacles, and four dirty locations. Coordinates are zero-based: the top-left cell is `(0,0)`. The charging station is at `(0,0)`.

The dirty locations are assigned task labels according to the order in which their coordinates appear in `Environment_Sample.txt`:

- `D1 = (0,7)`
- `D2 = (3,7)`
- `D3 = (7,7)`
- `D4 = (7,0)`

The following grid illustrates the defined environment.

	0	**1**	**2**	**3**	**4**	**5**	**6**	**7**
0	C							D1
1				X				
2				X				
3				X				D2
4				X				
5				X				
6				X				
7	D4							D3

Grid: 8 x 8
`C` = charging station
`D1`, `D2`, `D3`, `D4` = dirty locations
`X` = obstacle
blank cell = free cell

The corresponding sample scheduling file `Schedule_Sample.txt` defines four time slots and the following constraints:

- `D1` must be cleaned before `D3`.
- `D2` must be cleaned before `D4`.
- `D1` must be cleaned before `D4`.
- `D3` must be cleaned no later than `T3`.
- `D2` must be cleaned no later than `T3`.

- At most one task may be assigned to each time slot.

One feasible schedule is:

Time slot	Cleaning task
T1	D1
T2	D2
T3	D3
T4	D4

The sample files are illustrative and may also be used to test the integrated program. The format of these files are discussed in the proceeding section.

4. Environment Requirements

Each student/group must construct a grid-based vacuum-cleaning environment (different from the sample environment) satisfying the following minimum requirements:

- **Grid size:** At least 8 × 8 cells.
- **Obstacles:** At least six obstacle cells.
- **Dirty locations:** At least four distinct dirty locations.
- **Charging station:** Exactly one charging station.
- **Robot:** The robot starts at the charging station.
- **Movement:** The robot can move up, down, left, or right only. Diagonal movement is not allowed.
- **Reachability:** Every dirty location must be reachable from the charging station through valid paths.
- **No overlap:** The charging station, dirty locations, and obstacles must occupy different cells.
- **Obstacle restriction:** The robot cannot move through obstacle cells.

Students may use a larger grid and/or include additional obstacles and dirty locations, provided that the environment remains solvable.

5. Graph-Based Search: Robot Navigation

5.1. Objective

Implement at least two graph-search algorithms to find a valid path between two specified locations in the environment while avoiding obstacles. Students must implement at least two graph-search algorithms covered in the AI course, including at least one uninformed and one informed search algorithm. Both algorithms must solve the same start-to-goal problem on the same environment.

5.2. Environment Input File

Each student/group must define an environment and store its specification in a text file named Environment.txt. The program must load and process this file rather than hardcoding the environment.

Use the following format, with one item or group of items per line:

1. Grid dimensions as rows,columns.
2. Number of obstacles.
3. Coordinates of all obstacles, represented as (row,column) tuples.
4. Number of dirty locations.
5. Coordinates of all dirty locations, represented as (row,column) tuples.
6. Coordinate of the charging station, represented as a (row,column) tuple.

Coordinates are zero-based. The first dirty location listed is D1, the second is D2, and so on.
Refer to `Environment_Sample.txt` for an example of the required format.

5.3. Problem Formulation

Students must formulate the navigation problem as follows:

- **Environment grid:** Represent the environment loaded from Environment.txt, identifying free cells, obstacles, dirty locations, and the charging station.
- **State:** Represent the robot's position as (row,column).
- **Initial state:** The starting position supplied to the search function.
- **Actions:** UP, DOWN, LEFT, and RIGHT. An action is valid only if the resulting cell is within the grid boundaries and is not an obstacle.
- **Transition model:** A valid action moves the robot to the corresponding neighboring cell.
- **Goal state:** The target position supplied to the search function.
- **Step cost:** Each valid movement has a cost of 1.

5.4. Requirements

- The start and goal positions must be provided as inputs rather than hardcoded in the search algorithm.
- The program must load the environment from `Environment.txt`.
- The same environment and start/goal pair must be solvable using each implemented search algorithm.
- Every returned path must contain valid, adjacent grid positions and must not pass through obstacles.
- The program must correctly handle cases where the start equals the goal or no path exists.
- For each algorithm, report the resulting path, path cost, and number of expanded nodes.
- Compare the implemented algorithms in terms of path cost, nodes expanded, and their behavior and performance.

5.5. Required Functions

Students must implement the following functions.

Successor Function - `get_successors`

Find all valid successor states without including cells outside the grid or obstacle cells.
Arguments:

- `state`: Current robot position.
- `grid`: Environment representation.

Returns:

- A list of valid successors, each represented as `(next_state, action, step_cost)`.

Goal Test - `goal_test`

Checks whether the goal has been reached.
Arguments:

- `state`: Current robot position.
- `goal_state`: Target position.

Returns:

- `True` if `state` equals `goal_state`; otherwise, `False`.

Path Cost - `path_cost`

Calculates the total cost of a path.
Arguments:

- `path`: Sequence of states from the start to the goal.

Returns:

- Total path cost.

Since each movement costs 1, the path cost equals the number of movements, or the number of states in the path minus one. An empty path and a path containing only the initial state must be handled appropriately.

Heuristic Function - `heuristic`

Estimate the cost from the current state to the goal.
Arguments:

- `state`: Current robot position.
- `goal_state`: Target position.

Returns:

- An estimate of the cost from the current state to the goal.

For four-directional movement, students may use Manhattan distance:

```
h(n) = |row_n - row_goal| + |col_n - col_goal|
```

Students must explain why their selected heuristic is appropriate for the environment and search algorithm.

Search Function – `search`

Implements the selected search algorithm.

Arguments:

- `grid`: Environment representation.
- `start`: Starting position.
- `goal`: Target position.
- `algorithm`: Selected graph-search algorithm.

Returns:

- `path`: Sequence of states from start to goal, or an appropriate indication that no path exists.
- `cost`: Total path cost, if a path is found.
- `nodes_expanded`: Number of nodes expanded during the search.

6. Constraint Satisfaction Problem (CSP): Cleaning Task Scheduling

6.1. Objective

Use a CSP to generate a feasible cleaning schedule for the robot while satisfying the scheduling constraints. Each dirty location specified in Environment.txt represents one cleaning task. Students must assign every task to an available cleaning time slot while satisfying the constraints defined in Schedule.txt.

6.2. CSP Formulation

- **Variables:** Each dirty location represents a CSP variable. Task labels `D1`, `D2`, and so on are assigned in the order in which dirty locations appear in `Environment.txt`.
- **Domains:** Each variable initially has the set of available time slots, {`T1`,`T2`,...,`Tn`}, as its domain. Domains may be reduced during constraint propagation.
- **Precedence constraint:** A specified task must be completed before another task. For example, `D1` before `D3` means that the time slot assigned to `D1` must precede the time slot assigned to `D3`.
- **Latest-slot constraint:** A task must be scheduled no later than a specified time slot. For example, `D3` by `T3` means `D3` must be assigned `T1`, `T2`, or `T3`.
- **Slot-exclusivity constraint:** At most one task may be assigned to each time slot.

Every dirty location must be assigned exactly one time slot. Because no two tasks may share a time slot, the number of available time slots must be at least the number of dirty locations. A schedule is valid only if every task is assigned and all constraints are satisfied.

6.3. Scheduling Input File

Students must define the available cleaning time slots and scheduling constraints in a separate text file named `Schedule.txt`. The program must load the scheduling problem from this file rather than hardcoding the time slots or constraints.

The file must contain the following information in the specified order:

1. Number of available time slots.
2. Number of precedence constraints.
3. One pair (task_before,task_after) per line for each precedence constraint.
4. Number of latest-slot constraints.
5. One pair (task,latest_slot) per line for each latest-slot constraint.

Slot exclusivity applies automatically to all tasks and does not need to be listed in the file.
Refer to `Schedule_Sample.txt` for an example of the required format.

6.4. Requirements

- The number of available time slots must be at least the number of dirty locations.
- Task labels used in the scheduling constraints must correspond to the dirty locations defined by Environment.txt.
- The program must support different valid scheduling files without relying on a fixed number of tasks, slots, or constraints.
- Students must define at least three precedence constraints and two latest-slot constraints.
- Every dirty location must be assigned exactly one time slot.
- No two dirty locations may be assigned the same time slot.
- All precedence and latest-slot constraints must be satisfied.
- The program must correctly report when no feasible schedule exists.
- The CSP variables must be created from the dirty locations in `Environment.txt`, including tasks that do not appear in any precedence or latest-slot constraint.
- The program must display the generated schedule and demonstrate that it satisfies all specified constraints. Students must explain their CSP formulation, including its variables, domains, constraints, and the role of each implemented function.

6.5. Required Functions

Students must implement the following functions.

Variable Selection - `select_unassigned_variable`

Selects an unassigned variable using the Minimum Remaining Values (MRV), also called the most-constrained-variable heuristic. If multiple variables are tied, uses the degree heuristic to select the variable connected to the largest number of other unassigned variables through constraints.

Arguments:

- `assignment`: Current variable assignments.
- `domains`: Current domains of the variables.
- `constraints`: Scheduling constraints.

Returns:

- The next unassigned variable to select.

Value Ordering - `order_domain_values`

Orders the candidate time slots for the selected variable, using Least Constraining Value (LCV) heuristic to order values.

Arguments:

- `variable`: Selected variable.
- `assignment`: Current partial assignment.
- `domains`: Current domains.
- `constraints`: Scheduling constraints.

Returns:

- An ordered list of candidate time slots.

Constraint Checking - `is_consistent`

Checks precedence, latest-slot, and slot exclusivity constraints whenever the necessary assignments are available.

Arguments:

- `variable`: Variable being assigned.
- `value`: Proposed time slot.
- `assignment`: Current partial assignment.
- `constraints`: Scheduling constraints.

Returns:

- `True` if the proposed assignment does not violate any constraint that can be evaluated using the current assignment; otherwise, `False`.

Forward Checking - `forward_check`
Removes time-slot values that are no longer feasible for related unassigned variables. If any unassigned variable has an empty domain, the current assignment is rejected. Any domain values removed during forward checking must be restored when backtracking to a previous assignment.
Arguments:

- `variable`: Newly assigned variable.
- `value`: Assigned time slot.
- `assignment`: Current assignment.
- `domains`: Current domains.
- `constraints`: Scheduling constraints.

Returns:

- Updated domains if the assignment is consistent and no domain becomes empty; otherwise, a failure indication.

Backtracking Search - `backtracking_search`
Implement a backtracking CSP solver using constraint checking, forward checking, and the required variable-selection heuristic.
Arguments:

- `variables`: CSP variables.
- `domains`: Initial domains.
- `constraints`: Scheduling constraints.

Returns:

- A complete assignment if a feasible schedule exists; otherwise, a failure indication.

The solver must use the required variable-selection and constraint-checking functions. It must perform forward checking after assigning a value and backtrack when an assignment leads to failure. Students may also use the optional LCV heuristic.

7. Integration

Integrate the independently implemented CSP and graph-based search components to execute a complete cleaning task.

7.1. Requirements

Students must integrate their CSP and graph-based search implementations into a single robot simulation.

1. **Generate the cleaning schedule:** Load the cleaning tasks from `Environment.txt` and the time slots and scheduling constraints from `Schedule.txt`. Use the CSP solver to generate a feasible assignment of cleaning tasks to time slots.
2. **Determine the cleaning order:** Arrange the dirty locations according to their assigned time slots to obtain the cleaning sequence.
3. **Navigate to cleaning locations:** Starting from the charging station, use the graph-based search component to find a valid path from the robot's current position to each dirty location in the scheduled order.
4. **Execute the cleaning sequence:** Simulate the robot's movement along the returned paths and mark each dirty location as cleaned when the robot reaches it.
5. **Return to the charging station:** After cleaning all dirty locations, use graph-based search to find a valid path back to the charging station.
6. **Handle failures:** If the CSP solver cannot find a feasible schedule or graph search cannot find a required path, the program must report the failure appropriately.

7.2. Required Function

Implement the following function.

Integration Function - `execute_schedule`
Executes the CSP-generated schedule using graph-based search.
Arguments:

- `grid`: Environment loaded from Environment.txt.
- `charging_station`: Position of the charging station.
- `schedule`: Cleaning tasks ordered according to their assigned time slots by the CSP solver.
- `algorithm`: Graph-search algorithm to use for navigation.

Returns:

- `complete_path`: Combined path followed by the robot, from the charging station through all scheduled cleaning locations and back to the charging station.
- `total_cost`: Total navigation cost of the complete journey.
- `success`: Indicates whether all dirty locations were cleaned and the robot returned to the charging station successfully.

Procedure:

1. Initialize the robot's current position to the charging station.
2. Obtain the cleaning locations in the order specified by the CSP-generated schedule.
3. For each cleaning location, call search using the robot's current position as the start state and the next cleaning location as the goal state.
4. Append each returned path to complete_path, avoiding duplication of shared endpoints.
5. Update the robot's current position and mark the reached location as cleaned.
6. After all cleaning locations have been visited, call search again to find a path back to the charging station.
7. Combine the paths and calculate the total navigation cost.
8. Report the complete journey, total cost, and execution status.
9. If a required path cannot be found, the program must report failure rather than claim that the cleaning task was completed successfully.

Output Requirements

The integrated program must display:

- The cleaning schedule generated by the CSP solver.
- The sequence of dirty locations visited by the robot.
- The path computed for each journey between locations.
- The final grid showing the completed cleaning task.
- The total number of movements and total navigation cost for the complete journey.
- The final status indicating whether all dirty locations were cleaned and the robot returned to the charging station.

The CSP and graph-based search components must remain independently implementable and testable. The integration component must use their outputs rather than replace either algorithm with hardcoded decisions.

8. Deliverables

8.1. Code Implementation

- Develop the complete implementation in a single Jupyter Notebook.
- Add a Markdown cell at the beginning of the notebook containing the names and roll numbers of all group members.
- Save the notebook using the following naming convention: CS24XXX.ipynb, where XXX represents the three-digit roll number of any one group member (e.g., CS24009.ipynb).
- Organize the implementation into clearly defined sections for environment loading, graph-based search, CSP scheduling, and integration.
- Implement the required functions specified in the project handout.
- Include comments and Markdown cells throughout the notebook to explain the implementation, algorithms, important design decisions, and results.
- Ensure that the program loads the environment and scheduling constraints from `Environment.txt` and `Schedule.txt`, respectively, rather than relying on hardcoded input data.
- Include the input files used for testing and ensure that the notebook can be executed using the submitted files.

8.2. Report

Prepare a report in PDF format organized as follows:

- **Problem Definition and Representation:** Explain the problem and its representation, including the environment grid, states, actions, transition model, goal state, step cost, CSP variables, domains, and constraints.
- **Graph-Based Search Algorithms:** Describe the two implemented search algorithms, including their working principles, pseudocode, and any heuristic used.
- **CSP Formulation and Implementation:** Explain the CSP formulation and the implementation of variable selection, degree heuristic, constraint checking, forward checking, backtracking, and the optional Least Constraining Value (LCV) heuristic, if implemented.
- **System Integration:** Explain how the CSP-generated schedule determines the cleaning order and how graph-based search is used to navigate between consecutive cleaning locations and return to the charging station.
- **Results and Comparative Analysis:** Present tabular and/or graphical comparisons of the two graph-search algorithms, including paths, path costs, and numbers of expanded nodes. Discuss differences in their performance and behavior.
- **Testing and Validation:** Perform at least three test runs as follows:
  1. Test the system using your own environment, as defined in `Environment.txt` and `Schedule.txt`.
  2. Test the system using the provided environment, as defined in `Environment_Sample.txt` and `Schedule_Sample.txt`.
  3. Modify the environment or scheduling constraints to demonstrate how the system handles at least one failure case, such as an unreachable destination or an infeasible schedule. Name these files as `Environment_Error.txt` and `Schedule_Error.txt`.

  Include screenshots showing the configuration and output of all three test runs in both the Jupyter Notebook and the project report. For the failure case, the screenshots must demonstrate how the system detects and reports the failure.
- **Discussion and Improvements:** Discuss the correctness and performance of the implemented algorithms, including completeness and optimality where applicable. Identify limitations and suggest possible improvements.
- **References:** List any books, papers, websites, or other resources used.

Generative AI Use Declaration

At the end of the report, include a short declaration about the use of Generative AI tools (e.g., ChatGPT, Copilot, Gemini). The declaration must clearly state:

- Whether any Generative AI tool was used during the project.
- The specific purpose for which it was used, such as understanding concepts, generating ideas, debugging code, or assisting with report writing.
- Confirmation that the group reviewed and understood its implementation and results and takes responsibility for the submitted work.

Report Formatting and Submission Notes

- Attach the provided assessment rubric at the beginning of the report, with the names and roll numbers of all group members filled in.
- No additional title page is required; the rubric sheet will serve as the title page.
- Use Times New Roman font, with font size 11 for regular text and 12 or 14 for headings.
- For online submission, save the report as CS24XXX.pdf, where XXX represents the three-digit roll number of any one group member (e.g., CS24009.pdf).
- For hardcopy submission, print the report double-sided, staple it neatly, and submit it without a plastic file.

9. Grading

The project carries 30 marks in total:

- 20 marks: Practical component (Final Examination).
- 10 marks: Theory component (Assignment category).

10. Submission

10.1. Online submission:

- Submit the Jupyter Notebook (.ipynb), the PDF report, and the required input files (`Environment.txt`, `Schedule.txt`, `Environment_Error.txt`, and `Schedule_Error.txt`) through Google Classroom.
- Follow the specified naming conventions for the notebook and report.
- The submission deadline is the end of Week 14, November 22, 2026.
- Any one group member may submit the files on behalf of the group.

10.2. Final practical examination:

- Bring a soft copy of the Jupyter Notebook and the required input files.
- Bring a hard copy of the report.
- Be prepared to demonstrate the implementation, explain the algorithms and CSP formulation, justify design decisions, and answer questions about the submitted code and results.